STUDENT TASK MANAGER AWS DEPLOYMENT GUIDE

Complete project explanation in English and Thanglish
Nginx - Gunicorn - Flask - VPC - ALB - Auto Scaling - RDS

Purpose

This guide explains the complete Student Task Manager project from the frontend screen to the AWS database. It is written for project demonstration, interviews, and viva preparation.

Use it like a story: first understand the application, then understand the request flow, and finally prepare with the 30 viva questions.

1. Project at a glance

What did you build?

English: A full-stack Student Task Manager. A student can register, log in, create tasks, edit them, mark them complete, delete them, and view dashboard statistics. The application is deployed on AWS.

Thanglish: Idhu oru full-stack Student Task Manager. Student register/login pannuvaanga; task add, edit, complete, delete pannuvaanga; dashboard-la summary paapanga. App AWS-la deploy pannirukku.

Frontend	HTML, CSS, JavaScript. It displays the dashboard and task-management screens.
Backend	Python Flask REST API. It validates requests and runs database operations.
Database	Amazon RDS MySQL. It keeps user and task data permanently.
Deployment	Nginx receives public HTTP traffic, Gunicorn runs Flask, EC2 hosts the app.
Scale and availability	Application Load Balancer, target group health checks, and Auto Scaling support multiple EC2 instances.

End-to-end request flow

Example: when a user marks a task as completed, the browser sends a PATCH request. It travels through the ALB and Nginx to Gunicorn/Flask. Flask updates the matching task in RDS MySQL and returns a success response. The frontend refreshes the task status.

2. Frontend and API layer

Frontend responsibilities

English: The frontend is the part the student sees. It provides the Register, Login, Dashboard, Tasks, and Profile pages. JavaScript handles form validation, button clicks, toast messages, filters, and API calls.

Thanglish: Frontend-na user paakura part. Register, Login, Dashboard, Tasks, Profile pages irukkum. JavaScript form validation, button click, toast message, filter, API call ellam handle pannum.

Why api.js matters

English: The API service keeps the backend address in one place. Instead of writing a server URL in every page, the frontend sends all REST calls through a centralized service.

Thanglish: api.js central connection file. Ella page-layum URL hardcode pannaama, ore place-la backend URL set pannuvom. Appuram all API calls adha use pannum.

Register	POST /api/register - creates a user with a hashed password.
Login	POST /api/login - verifies credentials and returns the user identity.
Tasks	POST, GET, PUT, DELETE /api/tasks - task CRUD operations.
Completion	PATCH /api/tasks//complete - changes the completed state.
Dashboard	GET /api/dashboard?user_id=... - returns totals and completion statistics.

Important data ownership rule

Every task has a user_id. The backend uses it to make sure one student sees and changes only their own tasks.

Thanglish: Oru task-ku user_id irukkum. Adhu nala oru student-oda task-ai vera student edit/delete panna mudiyadhu.

3. Flask - Gunicorn - Nginx explained simply

Flask: the application brain

English: Flask is the Python web framework where your API code lives. It receives a request, validates data, connects to MySQL, performs the work, and sends JSON back.

Thanglish: Flask-na app-oda brain. Request vandha data validate pannum, MySQL-kitta pesum, work complete pannitu JSON response return pannum.

Gunicorn: the production runner

English: Gunicorn is a WSGI server. It runs your Flask application using production worker processes. Flask alone has a development server, which is suitable for learning but not for public deployment.

Thanglish: Gunicorn-na Flask app-a production-la run panna use pannura server. Flask-oda built-in server learning/testing-ku okay; public production-ku Gunicorn better.

Nginx: the public gatekeeper

English: Nginx is the web server and reverse proxy. It receives public traffic on port 80 or 443, can serve static files, and forwards API requests internally to Gunicorn. Gunicorn does not need to be directly exposed to the internet.

Thanglish: Nginx-na public gatekeeper. Browser port 80/443-la request anuppum. Nginx adha internal-a Gunicorn-kitta forward pannum. Gunicorn direct-a internet-ku expose aaga vendam.

Nginx asks	Where should this HTTP request go? It routes it to the backend safely.
Gunicorn asks	Which worker process can run this Flask application request?
Flask asks	Is this input valid? Which database query is required? What JSON should I return?

Easy analogy

Nginx is the reception desk. Gunicorn is the office manager who gives work to employees. Flask is the employee who understands the task and talks to the database.

4. AWS network and security design

VPC

English: A Virtual Private Cloud is your isolated AWS network. It defines the private address range and contains subnets, route tables, gateways, and security groups.

Thanglish: VPC-na AWS-kulla unga own private network. Idhula subnet, route table, gateway, security group ellam irukkum.

Public and private subnets

English: The public subnets hold the ALB because it must receive internet traffic. Application and database resources should be protected by security groups; RDS is not publicly accessible.

Thanglish: Public subnet-la ALB internet traffic receive pannum. Application and database security group protect pannum; RDS public-a irukka koodadhu.

ALB security group	Allows public HTTP/HTTPS traffic.
App security group	Allows application traffic from the ALB security group, plus restricted SSH administration.
Database security group	Allows MySQL port 3306 only from the application security group.
Target group	Contains EC2 instances and uses health checks to route only to healthy instances.
Auto Scaling Group	Maintains the desired number of EC2 instances and can grow or shrink within set limits.

Why this is safer

The database is not open to the public internet. A request must pass through the ALB and application layer before Flask can access RDS.

5. Step-by-step: one task completion request

1. User action

English: The student clicks Complete on a task in the browser.

Thanglish: Student task-la Complete button click pannuvanga.

2. Frontend request

English: JavaScript sends PATCH /api/tasks//complete with user_id and completed=true.

Thanglish: JavaScript PATCH request anuppum; adhula user_id and completed=true irukkum.

3. ALB routing

English: The ALB accepts the request and selects a healthy EC2 target from the target group.

Thanglish: ALB request receive panni healthy EC2 instance-a choose pannum.

4. Nginx proxy

English: Nginx receives HTTP traffic on the EC2 instance and proxies it to Gunicorn.

Thanglish: Nginx port 80-la request receive panni Gunicorn-kitta forward pannum.

5. Gunicorn worker

English: Gunicorn gives the request to a Python worker running the Flask app.

Thanglish: Gunicorn Python worker-ku request kudukkum; andha worker Flask app run pannum.

6. Flask logic

English: Flask validates user ownership and updates the task completion field.

Thanglish: Flask user ownership verify panni completed value update pannum.

7. RDS persistence

English: MySQL stores the update permanently in Amazon RDS.

Thanglish: Amazon RDS MySQL-la permanent-a update save aagum.

8. Response

English: Success JSON returns through the same path. The UI refreshes the status.

Thanglish: Success JSON same route-la browser-ku return aagum; UI status refresh aagum.

6. Interview and viva questions with answers

Use these answers as a starting point. In a viva, explain using your own project flow rather than memorising every sentence.

1. What is your project?

A full-stack Student Task Manager deployed on AWS. Users manage tasks through a frontend, Flask APIs, and an RDS MySQL database.

2. What problem does it solve?

It helps students organize assignments, priorities, due dates, and completion progress in one place.

3. What is the frontend technology?

HTML, CSS, and JavaScript, with a centralized API service for backend communication.

4. What is Flask?

Flask is a lightweight Python web framework used to build REST API endpoints and business logic.

5. Why did you use Flask?

It is simple, lightweight, easy to connect with MySQL, and suitable for a student REST API project.

6. What is Gunicorn?

Gunicorn is a production WSGI server that runs the Flask application with worker processes.

7. Why not expose Flask directly?

Flask development server is not designed for production. Gunicorn and Nginx provide a safer, more reliable deployment path.

8. What is Nginx?

Nginx is a high-performance web server and reverse proxy that receives public HTTP/HTTPS traffic and forwards API requests internally.

9. What is a reverse proxy?

It is a server in front of an application server. Clients talk to Nginx, while Nginx passes requests to Gunicorn/Flask behind it.

10. What is the request flow?

Browser -> ALB -> EC2 Nginx -> Gunicorn -> Flask -> RDS MySQL, then the response travels back.

11. What is REST API?

A REST API exposes resources through HTTP methods such as GET, POST, PUT, PATCH, and DELETE.

12. Which HTTP methods do you use?

POST for registration and task creation, GET for reading, PUT for editing, PATCH for completion, and DELETE for removal.

13. What is Amazon EC2?

EC2 provides virtual servers where Nginx, Gunicorn, and Flask run.

14. What is Amazon RDS?

RDS is AWS managed relational database service. In this project it hosts MySQL.

15. Why use RDS instead of local MySQL?

RDS handles managed backups, maintenance, availability options, and keeps database operations inside AWS.

16. What is a VPC?

A VPC is an isolated virtual network in AWS where subnets, route tables, security groups, EC2, and RDS are organised.

17. What is the difference between public and private subnets?

Public subnets can route to the internet through an Internet Gateway. Private resources are protected from direct internet exposure.

18. What is an Internet Gateway?

It connects a VPC public route to the internet, allowing public resources such as an internet-facing ALB to receive traffic.

19. What is a security group?

A stateful virtual firewall that controls inbound and outbound network traffic for AWS resources.

20. How is the database protected?

The RDS security group allows MySQL traffic only from the application security group, not from the public internet.

21. What is an ALB?

An Application Load Balancer distributes HTTP/HTTPS requests across healthy application targets.

22. What is a target group?

A target group is the list of EC2 instances that the ALB can route requests to, together with health-check settings.

23. What is a health check?

It is a periodic request used by the ALB to verify that an EC2 application instance is healthy before sending user traffic.

24. What is Auto Scaling?

Auto Scaling maintains or adjusts the number of EC2 instances based on configured desired capacity and limits.

25. How do users own tasks?

Each task is stored with user_id. The API uses user_id during read, update, completion, and delete operations.

26. How do you secure passwords?

Passwords are hashed before storage. The backend verifies a submitted password against its stored hash during login.

27. What is CORS?

Cross-Origin Resource Sharing is a browser security mechanism. Flask-CORS allows an approved frontend origin to call the API.

28. What happens if one EC2 instance fails?

The ALB health check marks it unhealthy and stops routing traffic to it. Another healthy target can continue serving requests.

29. What would you improve next?

Use HTTPS with ACM, restrict CORS, store secrets in AWS Secrets Manager, add proper token authentication, and enable CloudWatch alarms.

30. What is the key learning from this project?

It combines frontend development, API design, database operations, AWS networking, deployment, security groups, load balancing, and scaling.

7. Final revision checklist

Application	Can explain register, login, task CRUD, completion, dashboard.
Backend	Can explain Flask routes, validation, JSON responses, password hashing.
Deployment	Can explain Nginx -> Gunicorn -> Flask in one sentence.
AWS	Can explain VPC, subnets, ALB, target group, Auto Scaling, security groups, and RDS.
Security	Can explain why RDS is private and why port 3306 is restricted.
Demo	Can demonstrate task creation, edit, completion, deletion, and dashboard updates.

One sentence project summary

I developed and deployed a full-stack Student Task Manager where a JavaScript frontend communicates with Flask REST APIs running behind Nginx and Gunicorn on EC2, with task data stored securely in Amazon RDS MySQL and traffic distributed through an Application Load Balancer.

Thanglish: Naan build pannadhu full-stack Student Task Manager. Frontend Flask API-kitta pesum; Nginx and Gunicorn EC2-la Flask-a production-la run pannum; task data RDS MySQL-la save aagum; ALB healthy EC2 instances-kku traffic distribute pannum.